

Report on

AUTOFORM AI

Submitted to:
University Institute of Computing

Submitted by:
Satyam Kumar(25MCA20346)
MCA, Chandigarh University



Chandigarh University, Mohali

BONAFIDE CERTIFICATE

Certified that this project report “AUTOFORM AI” is the bonafide work of “Satyam” who carried out the project work under my/our supervision.

SIGNATURE
HEAD OF THE DEPARTMENT
(University Institute of Computing)

SIGNATURE
SUPERVISOR
(Assistant Professor)

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER

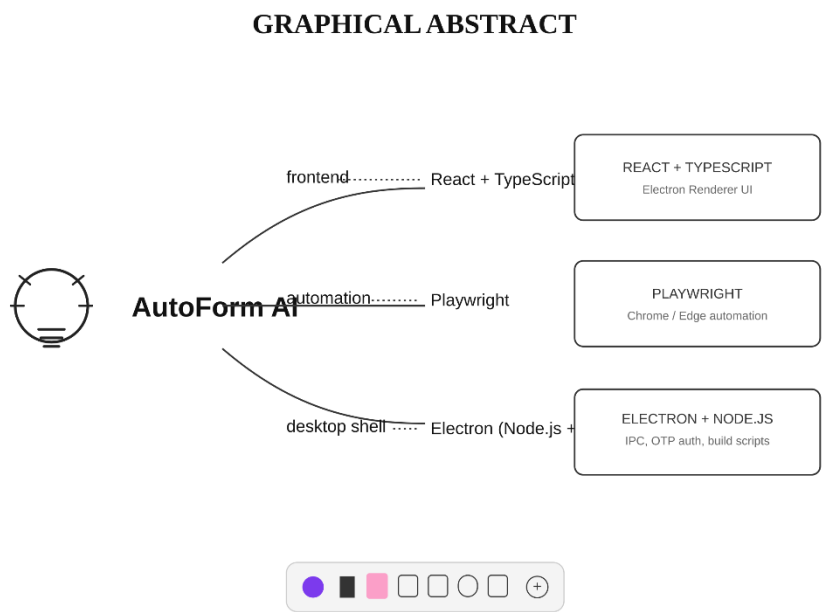
EXTERNAL EXAMINER

TABLE OF CONTENT

CHAPTER 1: INTRODUCTION	1
1.1 Need Identification.....	1
1.2 Identification of Problem	2
1.3 Identification of Tasks	3
1.4 Timeline	4
1.5 Organization of the Report.....	5
CHAPTER 2: BACKGROUND STUDY	6
2.1 Timeline of the Reported Problem.....	6
2.2 Proposed Solution	7
2.3 Bibliometric Analysis	8
2.4 Review Summary.....	9
2.5 Problem Definition.....	10
2.6 Goals / Objectives	10
2.7 Why Use These Technologies Over Others	11
CHAPTER 3: DESIGN AND PROCESS.....	12
3.1 Evaluation & Selection of Specifications/Features.....	12
3.2 Design Constraints	13
3.3 Design Flow	14
3.4 System Architecture.....	15
3.5 Core Modules.....	16
3.6 Implementation Plan / Methodology.....	17
CHAPTER 4: RESULT ANALYSIS AND VALIDATION	18
4.1 Implementation of Solution	18
4.2 Validation.....	19
CHAPTER 5: CONCLUSION AND FUTURE WORK	20
5.1 Conclusion	20
5.2 Future Work	21
REFERENCES	22

GRAPHICAL ABSTRACT

Fig 1:



AutoForm AI — technology overview (frontend, automation, desktop shell)

ABSTRACT

AutoForm AI is an intelligent desktop application designed to automate the process of filling online forms using a user-defined profile. The system helps reduce manual effort, improve efficiency, and minimize repetitive data entry across job applications, registration forms, surveys, and other web-based forms. The project combines desktop application development, browser automation, and intelligent field matching into a single solution.

Manually filling repetitive web forms is time-consuming and error-prone, and users often need to enter the same personal, academic, and professional details repeatedly across multiple websites and application portals. AutoForm AI addresses this problem by enabling users to store their information once, in a locally-managed profile, and reuse it automatically whenever a form needs to be filled.

The application is built as a cross-platform desktop tool using Electron, with a modern user interface developed in React and TypeScript, bundled with Vite. Browser automation and form detection are powered by Playwright, which allows the application to launch a Chrome or Edge session, analyze the structure of a webpage, detect input fields, text areas, dropdowns and choice-based elements, and populate them using the saved profile data. Communication between the Electron main process and the React renderer is handled through a secure IPC layer, and the system additionally provides password-based login with OTP verification through email for account recovery.

The current version of AutoForm AI is a working prototype (MVP) that successfully demonstrates profile management, browser-based form detection, autofill, and password recovery. The report documents the problem being addressed, the background research and technology choices, the design and architecture of the system, the working methodology, validation performed on the current build, and the future scope of the project — including AI-based field recognition, cloud synchronisation, and multi-user support.

CHAPTER 1

INTRODUCTION

1.1 Need Identification

In the present digital-first world, individuals are required to interact with a very large number of online forms as part of their everyday academic, professional, and personal activities. Job portals, college admission systems, government e-services, surveys, and registration pages on almost every website all demand that a user re-enter the same set of personal details — name, contact information, education, skills, and work experience — again and again.

This repetitive data entry is not only tedious but also a significant drain on time and attention, particularly for users such as job seekers and students who may need to apply to dozens of positions or forms within a short span of time. It also introduces a higher probability of human error — typos, missed fields, inconsistent formatting — which can, in some cases, affect the outcome of an application.

While password managers and browser-based autofill tools exist, they are generally limited to simple, previously-seen fields and do not offer intelligent, context-aware matching for the wide variety of field labels and structures used across different websites. There is a clear need for a dedicated, intelligent desktop tool that can understand the structure of a form, match it against a rich user profile, and fill it automatically — while still keeping the user in control of their data.

It is this need that led to the conceptualisation of AutoForm AI — a desktop application that stores a user's profile once and reuses it intelligently to automate form filling across the web.

1.2 Identification of Problem

A detailed review of the way users currently interact with online forms highlights the following specific problems:

- Users must manually re-enter identical personal, academic, and professional information across many different websites and portals.
- Manual entry is time-consuming, especially when a user is applying to multiple opportunities (jobs, colleges, events) within a short period.
- Repetitive typing increases the likelihood of inconsistent or incorrect data being submitted.
- Existing browser autofill features are generic, are not tailored to complex or dynamically generated forms, and offer little control or learning capability.
- There is no lightweight, desktop-based tool that combines profile management with genuine browser automation and field-level intelligence.

These problems collectively result in lost productivity and a frustrating user experience, particularly for people who repeatedly fill similar categories of forms, such as students applying for internships or job seekers applying to multiple companies.

1.3 Identification of Tasks

To convert the concept of AutoForm AI into a working application, the project was broken down into a series of well-defined tasks, each targeting a specific layer of the system:

1.3.1 Desktop Application Development (Electron)

- Setting up an Electron application shell responsible for window creation and application lifecycle.
- Establishing a secure preload script and IPC channel between the main process and the renderer.

1.3.2 Frontend Development (React + TypeScript)

- Building a clean, user-friendly interface for profile creation and management.
- Building login and OTP-based password recovery screens.
- Building controls to launch and monitor the automation session.

1.3.3 Browser Automation (Playwright)

- Launching a Chrome or Edge browser session from within the desktop application.
- Detecting form fields, text areas, dropdowns, and choice-based elements on the currently opened webpage.
- Matching detected fields to the stored profile using label and contextual rules, and auto-filling the matched values.

1.3.4 Authentication Module

- Implementing a password-based login system.
- Implementing OTP generation and delivery via email (using nodemailer) for password recovery.

1.3.5 Testing and Packaging

- Testing the autofill logic against a range of real-world form layouts.
- Building the Electron distributable using custom build scripts and ESBUILD.

1.4 Timeline

AutoForm AI has been developed in a series of iterative stages that moved from foundational setup through to a working MVP:

- Stage 1: Requirement analysis, problem identification, and technology stack selection (Electron, React, TypeScript, Vite, Playwright).
- Stage 2: UI design and development of the profile management and authentication screens.
- Stage 3: Implementation of the Electron main process, preload script, and IPC communication layer.
- Stage 4: Development of the Playwright-based browser automation and form detection module.
- Stage 5: Integration of field-matching logic to connect detected fields with profile data, and implementation of autofill.
- Stage 6: Implementation of OTP-based password recovery using nodemailer.
- Stage 7: Testing, bug fixing, and build packaging of the MVP.
- Stage 8: Documentation and planning for future enhancements (AI-based field recognition, cloud sync).

1.5 Organization of the Report

This report is organised into five chapters, each addressing a specific aspect of the AutoForm AI project:

- Chapter 1 – Introduction: presents the need, problem statement, identified tasks, and project timeline.
- Chapter 2 – Background Study: discusses the evolution of the problem, the proposed solution, a bibliometric review of related work, and the rationale for the chosen technology stack.
- Chapter 3 – Design and Process: covers feature selection, design constraints, system architecture, core modules, and the implementation methodology.
- Chapter 4 – Result Analysis and Validation: reports on testing performed, the current status of the system, and the validation rules applied to user input.
- Chapter 5 – Conclusion and Future Work: summarises the outcomes of the project and outlines the roadmap for future versions.

CHAPTER 2

BACKGROUND STUDY

2.1 Timeline of the Reported Problem

2.1.1 Early Web Forms (Pre-2010s)

Online forms were largely static and simple, and the volume of forms an individual had to fill in daily was relatively small, so manual entry was not a significant burden.

2.1.2 Rise of Online Applications (2010–2018)

The growth of job portals, online college admissions, and e-governance services dramatically increased the number of forms a typical user needed to complete. Browsers began offering basic autofill for simple fields such as name, address and email.

2.1.3 Explosion of Digital Applications (2019–2022)

Remote work, online education and digital-first hiring during and after the pandemic multiplied the number of portals a single user interacted with. Job seekers in particular began applying to a large number of positions through different Applicant Tracking Systems, each requiring the same information to be entered afresh.

2.1.4 Current Scenario (2023–Present)

Users today routinely fill dozens of similar but structurally different forms. Basic browser autofill remains limited to a small set of recognised field types and does not generalise well to complex, dynamically rendered, multi-step forms. This has created a clear opportunity for a dedicated, intelligent, desktop-based autofill solution — the gap that AutoForm AI is designed to fill.

2.2 Proposed Solution

AutoForm AI proposes a desktop application that stores a user's profile information locally and uses genuine browser automation — rather than simple browser-extension autofill — to interact with online forms. Once the user provides the URL of a target form, the application launches a Chrome or Edge session, analyses the structure of the page, detects the available form components, and maps them against the saved profile before automatically populating the matching fields.

2.2.1 Profile Management

A dedicated profile module allows the user to enter and update personal, academic, and professional information once, including name, contact details, address, education, skills, experience, and social/professional links. This profile becomes the single source of truth used across all future autofill sessions.

2.2.2 Intelligent Field Detection and Matching

Rather than relying on a fixed list of known fields, the automation layer inspects the labels, placeholders, and surrounding context of each field on the page to decide which profile value, if any, should be inserted — giving the system a level of flexibility that generic browser autofill does not offer.

2.2.3 Authentication and Recovery

A password-based login system, combined with OTP verification delivered by email, secures access to the stored profile and protects the user's personal data.

2.3 Bibliometric Analysis

To understand the existing body of work around form automation, robotic process automation (RPA), and browser-based automation tools, a short review of literature and industry material relevant to this domain was carried out.

2.3.1 Data Sources

- Google Scholar
- IEEE Xplore
- Official documentation of automation frameworks (Playwright, Selenium, Puppeteer)

2.3.2 Key Findings

Existing literature on robotic process automation (RPA) largely focuses on enterprise-scale automation of repetitive back-office tasks, rather than on lightweight, personal-productivity tools aimed at individual users filling online forms. Browser automation frameworks such as Playwright and Selenium are well documented for testing and scraping use cases, but comparatively little published material addresses their application to personal form-autofill assistants of the kind AutoForm AI implements.

2.3.3 Literature Gap Identified

There is limited published work that combines (a) a locally-managed personal profile store, (b) contextual, label-based field matching rather than fixed-field autofill, and (c) a desktop-native automation layer built on a modern browser automation framework, into a single consumer-facing tool. AutoForm AI is positioned to address this specific combination.

2.4 Review Summary

The review of the problem space and existing tools confirmed that while generic browser autofill and enterprise RPA tools both exist, neither adequately serves an individual user who needs a personal, intelligent, desktop-based form-filling assistant. This gap directly informed the design of AutoForm AI, which combines a locally stored profile, contextual field matching, and genuine browser automation through Playwright, wrapped inside a modern Electron and React-based desktop application.

2.5 Problem Definition

Users routinely need to enter the same personal, academic, and professional information across a large number of unrelated web forms. Existing solutions — generic browser autofill and enterprise RPA platforms — do not adequately address the needs of an individual user who wants a lightweight, intelligent, desktop-native tool capable of understanding a form's structure and filling it accurately from a locally stored profile.

2.6 Goals / Objectives

- To reduce the time required for filling repetitive forms.
- To improve accuracy and consistency in data entry across different websites and portals.
- To provide a user-friendly interface for managing personal and professional profile information.
- To automate browser-based form interactions using modern automation tools such as Playwright.
- To support future enhancement through AI-based field recognition and smarter matching.

2.7 Why Use These Technologies Over Others

2.7.1 Electron for the Desktop Shell

Electron was selected because it allows the reuse of web development skills (HTML, CSS, JavaScript/TypeScript) to build a cross-platform desktop application capable of deep integration with the operating system and, importantly, capable of controlling a real browser instance through Playwright — something a purely web-based application cannot do.

2.7.2 React and TypeScript for the Interface

React was chosen for its component-based architecture and strong ecosystem, while TypeScript adds static type safety, which is particularly valuable when passing structured profile data between the renderer, the main process, and the automation layer.

2.7.3 Vite for the Build Pipeline

Vite was selected over older bundlers such as Webpack for its significantly faster development server start-up and hot-module-replacement, which sped up iteration on the renderer UI.

2.7.4 Playwright for Browser Automation

Playwright was chosen over alternatives such as Selenium or Puppeteer because of its modern API, reliable auto-waiting behaviour, and first-class support for both Chrome and Edge (Chromium-based browsers), which directly matches the project's requirement to automate the two most commonly used browsers.

2.7.5 Node.js and nodemailer for OTP Delivery

Node.js powers the main-process logic, and nodemailer was used to implement OTP-based email delivery for password recovery with minimal additional infrastructure.

CHAPTER 3

DESIGN AND PROCESS

3.1 Evaluation & Selection of Specifications/Features

A comparative look at existing autofill and RPA tools was carried out to identify feature gaps before finalising the feature set of AutoForm AI. The following categories of features were shortlisted for evaluation: profile management, browser automation, field detection and matching, authentication/security, and future AI-based enhancement.

3.1.1 Selected Key Features for the MVP

- Desktop application built with Electron.
- Modern user interface developed using React and TypeScript.
- Support for Chrome and Edge browser automation via Playwright.
- Form field detection from the currently opened webpage.
- Automatic autofill of detected and matched values.
- Saving and reusing user profile information locally.
- Learning of newly detected fields for future use.
- Password-based login system with OTP-based password reset through email.
- Support for text inputs, dropdowns, radio/choice groups, and file-upload-like elements.

3.2 Design Constraints

3.2.1 Technical Constraints

Because AutoForm AI drives a real browser instance through Playwright, its reliability is tied to the structure and behaviour of each target website. Highly dynamic, JavaScript-heavy forms, or forms that render fields only after specific user interaction, are more difficult to detect reliably than static forms.

3.2.2 Platform Compatibility Constraints

The application currently targets Windows and macOS desktop environments and automates the Chrome and Edge browsers specifically, since these are the most widely used Chromium-based browsers among the target users.

3.2.3 Budget Constraints

As a student-led project, the technology stack was chosen to minimise cost: Electron, React, TypeScript, Vite, and Playwright are all open-source and free to use, and nodemailer works with standard email accounts rather than a paid transactional-email service.

3.2.4 Privacy and Security Constraints

Since the application stores personal profile data, particular care was taken around local storage of this information, password-based authentication, and OTP verification for recovery. Future versions are planned to add stronger encryption for data at rest, in line with the project's non-functional requirements.

3.3 Design Flow

The design flow of AutoForm AI follows a straightforward user journey: the user logs in (or recovers their password via OTP), creates or updates their profile, provides the URL of a target form, and triggers the automation session. The application then opens the page in a controlled browser instance, detects the form fields, matches them to the profile, fills the matching fields, and reports back to the user on how many fields were filled or skipped.

3.3.1 Working Methodology

- The user enters or updates their profile information through the application interface.
- The profile data is stored locally for future reuse.
- The user provides a URL of a web form.
- The application launches the browser and opens the target page.
- The system analyzes the page structure and detects form fields.
- The detected fields are matched against the stored profile using contextual and label-based rules.
- Matching values are inserted into the corresponding form controls.
- The system provides feedback about the number of fields filled or skipped.

3.4 System Architecture

AutoForm AI follows a modular architecture consisting of the following layers:

- Frontend Layer: React-based UI for user interaction (profile entry, login, automation controls).
- Main Process Layer: Electron process responsible for application lifecycle and browser integration.
- Automation Layer: Playwright-based logic for interacting with browser pages.
- IPC Layer: Communication between the renderer and main processes.
- Authentication Layer: Password login and OTP recovery services.

3.5 Core Modules

3.5.1 User Interface Module

Handles profile entry, login, password reset, and autofill actions.

3.5.2 Profile Management Module

Stores user data such as name, email, phone number, address, education, skills, experience, and social links.

3.5.3 Browser Automation Module

Opens the browser, navigates to the desired URL, detects form elements, and fills them.

3.5.4 Form Detection and Matching Module

Analyzes field labels, contexts, and surrounding page structure to identify suitable values.

3.5.5 Authentication Module

Implements login and password recovery using OTP verification.

3.5.6 Important Project Files

- `src/main/main.ts` — Main Electron application entry point responsible for window creation and app lifecycle.
- `src/renderer/App.tsx` — Main UI component containing profile management, login flow, and automation controls.
- `src/main/automation/browser.ts` — Core implementation for browser launching, field detection, and autofill behaviour.
- `src/main/ipc/browser.ts` — Handles communication between the renderer process and main process.
- `src/main/preload.ts` — Provides secure access to Electron APIs from the renderer process.
- `src/main/otpService.ts` — Implements OTP generation and email-based password reset functionality.
- `scripts/build-electron.mjs` — Responsible for building the Electron application artefacts.

3.5.7 User Interface Screens

The following screenshots illustrate the primary screens of the AutoForm AI desktop application.

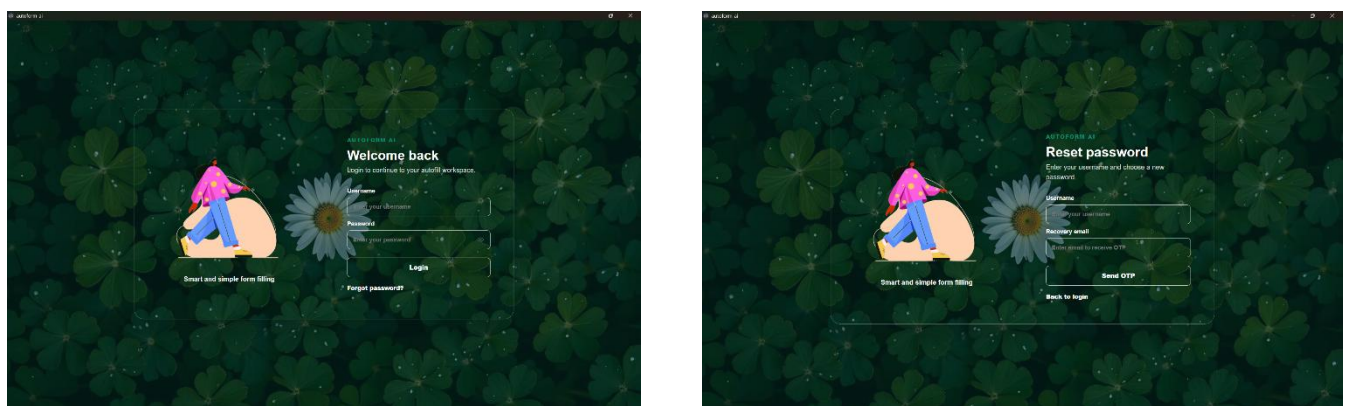


Fig 4: Login screen with OTP-based password recovery option

The image shows a screenshot of the 'Profile management' screen in the AutoForm AI application. The screen is split into two panels. The left panel has a dark background with a mountain landscape and the text 'Discover forms without the busywork.' Below this, it says 'Save profile data first, then scan a form' and 'Hint: the saved profile will be matched with incoming forms to start reports.' The right panel is a light blue form titled 'Personal details for the student'. It contains various input fields for personal and academic information: Full Name, First Name, Last Name, Mobile Number, Email, Date of Birth, Gender, Address, City, State, Country, Postal Code, and PIN. There are 'Clear' and 'Save Profile' buttons at the bottom right of the form.

Profile management screen (personal, academic, and professional details)

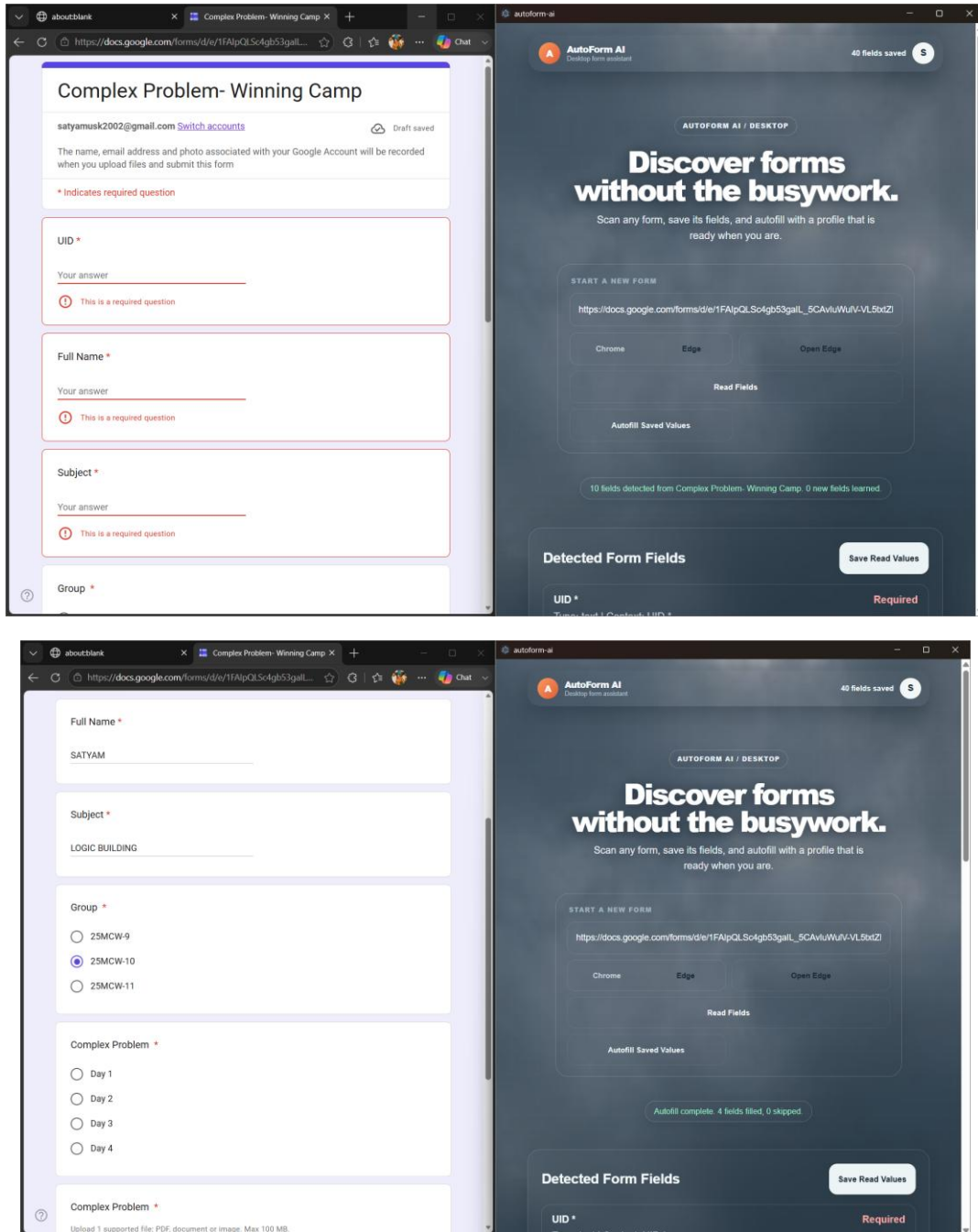


Fig 6: Automation control panel — entering the target form URL and starting a session

3.6 Implementation Plan / Methodology

The implementation of AutoForm AI followed an iterative, module-by-module methodology, allowing each layer of the system to be built and tested independently before integration.

Phase 1: Planning and Requirement Analysis

Defined the vision of the application, identified the target users (job seekers, students, professionals filling repetitive forms), and finalised the technology stack.

Phase 2: UI/UX Design

Designed the profile management, login, and automation-control screens using React, focused on clarity and ease of data entry.

Phase 3: Electron Application Shell

Set up the Electron main process, preload script, and secure IPC channel between the renderer and the main process.

Phase 4: Browser Automation Development

Implemented the Playwright-based automation module to launch Chrome/Edge, detect form fields, and perform autofill.

Phase 5: Field Matching Logic

Built the rule-based logic that maps detected field labels and context to the correct profile attribute.

Phase 6: Authentication and OTP Recovery

Implemented password-based login and the nodemailer-based OTP password-recovery flow.

Phase 7: Testing and Packaging

Manually tested the autofill flow across a range of form layouts, fixed identified issues, and packaged the Electron build using the custom build scripts.

CHAPTER 4

RESULT ANALYSIS AND VALIDATION

4.1 Implementation of Solution

The current build of AutoForm AI is a working prototype (MVP) that demonstrates the core end-to-end flow: profile creation, browser launch, form field detection, autofill, and password recovery via OTP. This section reviews how the implementation was assessed.

4.1.1 Report Preparation

Progress has been documented through each stage of development — from the initial feature evaluation and architecture design, through implementation of the automation and authentication modules, to the current MVP build — to provide a clear record of what has been completed and what remains for future versions.

4.1.2 Project Management and Communication

Development was organised around prioritising the core autofill flow first (profile → browser launch → field detection → autofill) before layering in authentication and recovery features. Work was tracked in phases, and this current report reflects the state of Version 1 of the project.

4.1.3 Testing, Characterization, and Interpretation

Manual testing was carried out across several representative form types — simple text-based forms, forms containing dropdowns and radio/choice groups, and multi-section forms — to observe how consistently the detection and matching logic performed. Testing also covered the login and OTP recovery flow to confirm that emails were correctly triggered and that recovery codes worked as expected.

The current limitations identified during testing include reduced accuracy on highly dynamic or unusually structured forms, and the current rule-based matching approach not generalising perfectly to every possible field label — both of which are noted as priorities in the future scope of the project.

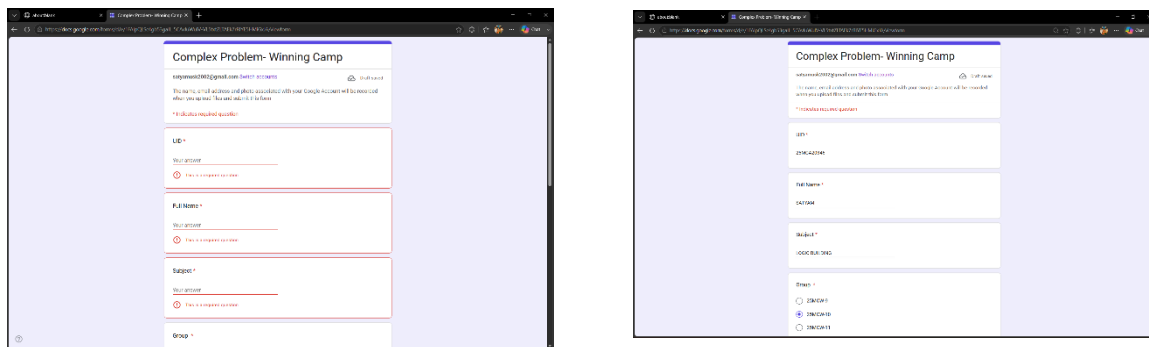


Fig 7: A sample web form before and after being auto-filled by AutoForm AI

4.2 Validation

Validation in AutoForm AI refers to the checks applied to ensure that profile data entered by the user, and the values used during autofill, are complete, correctly formatted, and safely handled.

4.2.1 Profile Data Validation (Frontend)

- Required profile fields (name, email, phone) must not be left empty before the profile can be saved.
- Email fields are checked for valid email format.
- Phone number fields are checked for valid numeric format and length.

4.2.2 Authentication Validation

- Login requires a valid registered email and matching password.
- Password fields enforce a minimum length.
- OTP codes are time-bound and are validated against the code generated and emailed to the user during password recovery.

4.2.3 Automation and Autofill Validation

- The target URL provided by the user is validated before the browser session is launched.
- Detected fields are cross-checked against the profile schema before a value is inserted, to avoid filling a field with mismatched data.
- The system reports the number of fields successfully filled versus skipped, allowing the user to verify the outcome of each automation run.

4.2.4 Planned Validation Enhancements

- Stronger, AI-assisted field-matching confidence scoring before auto-filling ambiguous fields.
- Encryption of locally stored profile data.
- Expanded validation for file-upload-type fields.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

AutoForm AI set out to answer a simple but genuinely time-consuming problem: why should a person re-enter the same personal, academic, and professional information again and again across dozens of unrelated web forms? By combining a locally managed user profile with genuine browser automation through Playwright, wrapped inside a modern Electron and React-based desktop application, AutoForm AI offers a practical answer to that problem.

The project has progressed from an initial problem identification and technology evaluation through to a working MVP that demonstrates the full core flow: profile management, browser-based form detection, contextual field matching, automatic form filling, and a secure, OTP-protected authentication system. While the current version relies on rule-based field matching and therefore does not yet handle every possible form layout perfectly, it successfully proves the feasibility of the underlying concept.

Technically, the use of Electron, React, TypeScript, Vite, and Playwright allowed the team to build a genuinely native-feeling desktop tool that can control a real browser session — something that goes well beyond what a simple browser extension or generic autofill feature can achieve. The modular architecture, separating the UI, automation, IPC, and authentication layers, also leaves the system well positioned for future enhancement.

In summary, AutoForm AI is a practical and innovative solution aimed at simplifying online form filling through intelligent automation. It offers a strong foundation on which more advanced AI-driven capabilities can be built in future versions.

5.2 Future Work

5.2.1 AI-Based Field Recognition

Replacing or augmenting the current rule-based matching logic with a machine-learning model capable of recognising field intent from label text, placeholder text, and surrounding page context, even for previously unseen field types.

5.2.2 Support for More Advanced and Dynamic Web Forms

Improving detection reliability for multi-step forms, forms rendered inside iframes, and forms that reveal fields only after specific user interactions.

5.2.3 Cloud-Based Profile Synchronisation

Allowing a user's profile to be securely synchronised across multiple devices, rather than being limited to local storage on a single machine.

5.2.4 Multi-User Authentication and Account Management

Extending the current single-user authentication system to support multiple accounts and role-based access, useful for future team or organisational use cases.

5.2.5 Export and Import of Profile Data

Enabling users to export their profile data (for backup or transfer) and import it back into the application.

5.2.6 Support for PDF and Document-Based Forms

Extending the autofill capability beyond web forms to PDF forms and other document-based application formats.

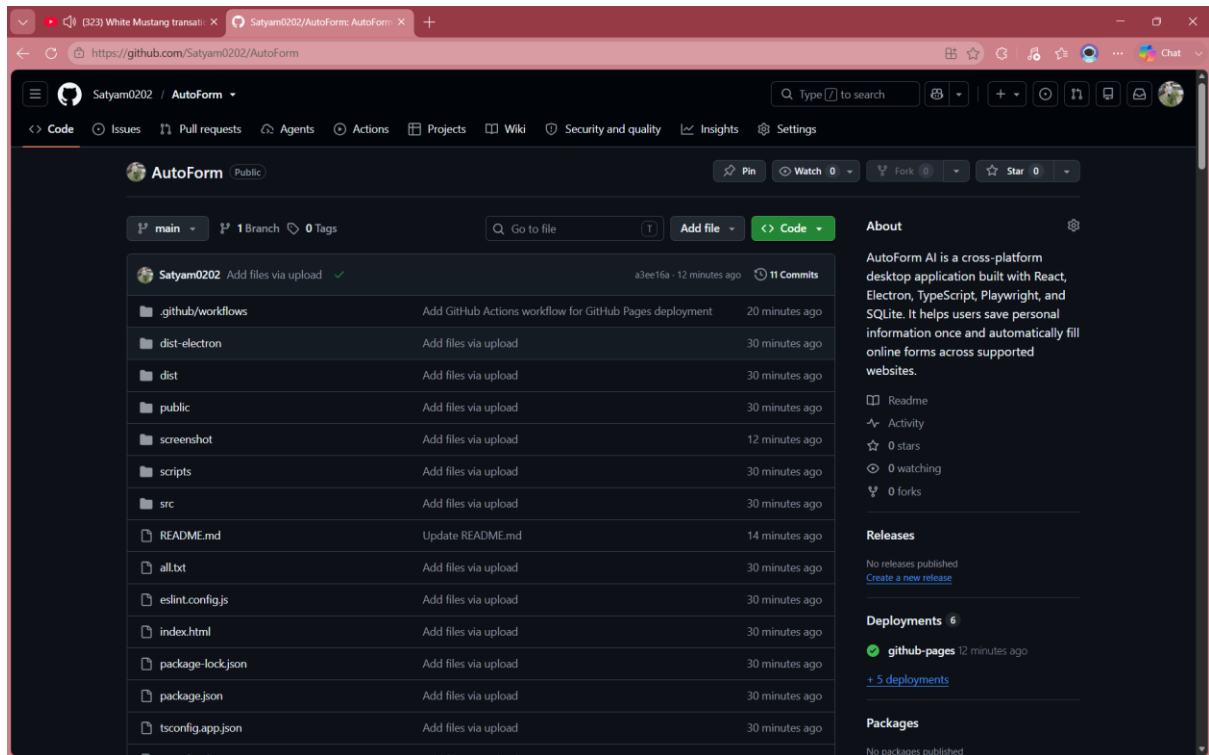
5.2.7 Mobile and Browser Extension Versions

Developing companion versions of AutoForm AI as a lightweight browser extension and, longer-term, a mobile application, to widen the contexts in which the tool can be used.

5.2.8 Stronger Security and Encryption

Adding encryption for locally stored profile data and strengthening the overall security posture of the authentication and OTP-recovery flows as the application matures beyond the MVP stage.

6. GitHub



[Satyam0202/AutoForm](https://github.com/Satyam0202/AutoForm)

REFERENCES

- [1] Playwright Documentation, Microsoft, <https://playwright.dev/>
- [2] Electron Documentation, OpenJS Foundation, <https://www.electronjs.org/docs/latest/>
- [3] React Documentation, Meta, <https://react.dev/>
- [4] TypeScript Documentation, Microsoft, <https://www.typescriptlang.org/docs/>
- [5] Vite Documentation, <https://vitejs.dev/>
- [6] Node.js Documentation, OpenJS Foundation, <https://nodejs.org/en/docs/>
- [7] nodemailer Documentation, <https://nodemailer.com/about/>
- [8] van der Aalst, W.M.P., Bichler, M. & Heinzl, A. Robotic Process Automation. *Bus Inf Syst Eng* 60, 269–272 (2018).